

1. What is the difference between \$* and @\$ in Bash?

Answer:

- \$* treats all arguments as a **single string**.
- @\$ treats all arguments as **separate words**.

Example:

```
for arg in "$*"; do echo $arg; done    # single iteration
for arg in "$@"; do echo $arg; done    # multiple iterations
```

2. How do you check if a file exists in Bash?

Answer:

Use -f for a regular file or -d for a directory.

```
FILE="/tmp/test.txt"

if [ -f "$FILE" ]; then
    echo "File exists"
else
    echo "File not found"
fi
```

3. How do you handle errors in Bash scripts?

Answer:

Use set -e to stop execution when a command fails.

```
#!/bin/bash
set -e
cp file1 file2    # script stops here if this fails
echo "✅ Completed successfully"
```

Or use manual handling:

```
command || { echo "❌ Command failed"; exit 1; }
```

4. How do you pass arguments to a Bash script?

Answer:

You can access arguments as \$1, \$2, etc.

```
#!/bin/bash
echo "First arg: $1"
echo "Second arg: $2"
```

Run as:

```
./script.sh dev prod
```

5. How do you read a file line by line in Bash?

Answer:

```
#!/bin/bash
while read -r line; do
    echo "Line: $line"
done < /etc/passwd
```

6. What is the purpose of `exit` and `$?` in Bash?

Answer:

- `exit` sets the script's return status.
- `$?` stores the exit status of the last command (0 = success, non-zero = error).

Example:

```
mkdir /root/test 2>/dev/null
if [ $? -ne 0 ]; then
    echo "Permission denied!"
fi
```

7. How do you use loops in Bash?

Answer:

```
for i in 1 2 3 4 5; do
    echo "Iteration $i"
done
```

Or with C-style:

```
for ((i=0; i<5; i++)); do
    echo "Count: $i"
done
```

8. How do you schedule a Bash script using Cron?

Answer:

Edit the crontab:

```
crontab -e
```

Add:

```
0 2 * * * /home/ec2-user/backup.sh >> /var/log/backup.log 2>&1
```

Runs every day at 2 AM.

9. How do you use functions in Bash scripts?

Answer:

```
greet_user() {  
    echo "Hello, $1!"  
}  
  
greet_user "Abhishek"
```

You can also return values:

```
add() {  
    echo $(( $1 + $2 ))  
}  
result=$(add 5 3)  
echo "Sum: $result"
```

10. What's the difference between `source script.sh` and `./script.sh`?

Answer:

- `source script.sh` runs the script in the **current shell** (variables persist).
 - `./script.sh` runs it in a **subshell** (variables don't persist).
-

11. How do you delete files older than 7 days?

Answer:

```
find /var/log -type f -mtime +7 -name "*.log" -exec rm -f {} \;
```

12. Write a Bash script to clean up unused Docker containers.

Answer:

```
#!/bin/bash  
echo "Cleaning unused containers..."  
docker container prune -f  
echo "Cleaning unused images..."  
docker image prune -a -f  
echo "✅ Cleanup done."
```

13. Write a Bash script to upload files to an AWS S3 bucket.

Answer:

```
#!/bin/bash
BUCKET="s3://myapp-backups"
FOLDER="/var/www/html"

aws s3 cp "$FOLDER" "$BUCKET" --recursive
if [ $? -eq 0 ]; then
    echo "✅ Upload successful"
else
    echo "❌ Upload failed"
fi
```

14. How do you get the top 5 CPU-consuming processes?

Answer:

```
ps -eo pid,comm,%cpu --sort=-%cpu | head -6
```

15. How do you check SSH connectivity to multiple servers?

Answer:

```
#!/bin/bash
HOSTS=("server1" "server2" "server3")

for host in "${HOSTS[@]}; do
    if ssh -o BatchMode=yes -o ConnectTimeout=5 "$host" "exit" 2>/dev/null; then
        echo "$host reachable"
    else
        echo "$host unreachable"
    fi
done
```

16. How do you use trap in Bash?

Answer:

trap lets you execute code when the script exits or gets interrupted.

```
#!/bin/bash
trap 'echo "⚠️ Script interrupted. Cleaning up..."; rm -f /tmp/tempfile' EXIT

touch /tmp/tempfile
echo "Running..."
sleep 5
```

17. How do you find and replace text in files using Bash?

Answer:

Using sed:

```
sed -i 's/oldtext/newtext/g' config.txt
```

18. How do you debug a Bash script?

Answer:

Run script in debug mode:

```
bash -x script.sh
```

Or add:

```
set -x # enable debug
set +x # disable debug
```

19. How do you extract a field from a file using awk?

Answer:

```
awk '{print $1, $3}' /etc/passwd
```

Prints the 1st and 3rd columns.

20. Write a script to restart all Kubernetes deployments in a namespace.

Answer:

```
#!/bin/bash
NAMESPACE=${1:-default}
for deploy in $(kubectl get deploy -n $NAMESPACE -o
jsonpath='{.items[*].metadata.name}'); do
    kubectl rollout restart deploy "$deploy" -n "$NAMESPACE"
done
```

How do you extract only the running processes using grep and ps?

Answer:

```
ps -ef | grep -v grep | grep "nginx"
```

This lists all running nginx processes excluding the grep process itself.

22. How can you count the number of lines in a file?

Answer:

```
wc -l filename.txt
```

Or inside a script:

```
LINES=$(wc -l < filename.txt)
echo "Total lines: $LINES"
```

23.How do you check if a variable is empty or not in Bash?

Answer:

```
if [ -z "$VAR" ]; then
    echo "Variable is empty"
else
    echo "Variable is set to: $VAR"
fi
```

-z returns true if the string length is zero.

24.How do you perform arithmetic in Bash?

Answer:

```
a=5
b=3
sum=$((a + b))
echo "Sum: $sum"
```

25.How do you get the IP address of your system in Bash?

Answer:

```
hostname -I | awk '{print $1}'
```

or

```
ip addr show eth0 | grep "inet " | awk '{print $2}' | cut -d/ -f1
```

26.Write a script to compress and archive log files.

Answer:

```
#!/bin/bash
tar -czf /backup/logs_$(date +%F).tar.gz /var/log/*.log
echo "Logs archived successfully"
```

27.How to measure the execution time of a script or command?

Answer:

```
start=$(date +%s)
sleep 3
end=$(date +%s)
echo "Execution time: $((end - start)) seconds"
```

28.How to print only unique lines from a file?

Answer:

```
sort filename.txt | uniq
```

29.How do you list all AWS EC2 instance IDs using Bash and AWS CLI?

Answer:

```
aws ec2 describe-instances --query "Reservations[*].Instances[*].InstanceId" --
output text
```

30.How to check if Docker is installed and running using Bash?

Answer:

```
if ! command -v docker &>/dev/null; then
    echo "Docker not installed"
    exit 1
fi

if ! systemctl is-active --quiet docker; then
    echo "Docker service not running"
    exit 1
fi

echo "Docker is up and running"
```

31.How do you get the nth column of a file using awk?

Answer:

```
awk '{print $3}' data.txt
```

This prints the third column from each line.

32.How do you iterate through a list of servers and check uptime?

Answer:

```
#!/bin/bash
SERVERS=("server1" "server2" "server3")

for host in "${SERVERS[@]}; do
    ssh "$host" uptime
done
```

33.How to send a Slack alert from Bash using curl?

Answer:

```
curl -X POST -H 'Content-type: application/json' \
--data '{"text":"High CPU usage on server"}' \
https://hooks.slack.com/services/SLACK_WEBHOOK_URL
```

34.Write a Bash script to find the top 5 memory-consuming processes.

Answer:

```
#!/bin/bash
ps -eo pid,comm,%mem --sort=-%mem | head -6
```

35.How do you extract a specific JSON value in Bash?

Answer:

If your JSON looks like this:

```
{"version": "1.2.3"}
```

You can extract the value using:

```
cat data.json | jq -r '.version'
```

36.How can you find the largest files in a directory?

Answer:

```
du -ah /var/log | sort -rh | head -5
```

37.How to check if a port is open on a remote server?

Answer:

```
if nc -z server1 22; then
    echo "Port 22 open"
else
    echo "Port 22 closed"
fi
```

38.Write a Bash script to check AWS CLI configuration.

Answer:

```
#!/bin/bash
if aws sts get-caller-identity >/dev/null 2>&1; then
    echo "AWS CLI configured correctly"
else
```

```
    echo "AWS CLI not configured"
fi
```

39.How can you rename multiple files at once in Bash?

Answer:

```
for file in *.log; do
    mv "$file" "${file%.log}_backup.log"
done
```

40.How to mask sensitive values like passwords in logs?

Answer:

```
cat app.log | sed 's/password=[^ ]*/password=*****/g'
```